

Regex Playbook

1. Introduction to Regular Expressions

What is Regex?

A **Regular Expression (Regex)** is a sequence of characters that defines a search pattern.

Uses of Regex

- Search text
- Validate user input
- Extract information
- Replace text
- Clean data

Example

```
pattern = "e"  
text = "Python Regex Playbook"
```

Matches first occurrence of e

2. Reading Regex Basics

Literal Characters

Match exactly what is written.

Example

```
regex = "sight"  
text = "Pluralsight"
```

✓ Match found

```
regex = "Sight"
```

✗ No match (case-sensitive)

3. Special Characters

Wildcard (.)

Matches any character except newline.

Example

```
regex = "f.n"
```

Matches:

fun

fan

fin

Example:

I'm a fan.

✓ Matches "fan"

Anchors (^ and \$)

Start Anchor (^)

```
regex = "^Plural"
```

Matches:

Pluralsight

End Anchor (\$)

```
regex = "sight$"
```

Matches:

Pluralsight

Escape Character ()

Used to search literal special characters.

Example

```
regex = r"regex\."
```

Matches:

I like regex.

Not:

I like regex!

4. Character Classes

Match One Character from a Group

```
[ae]
```

Matches:

a

e

Character Ranges

```
[a-z]
```

Lowercase letters

```
[A-Z]
```

Uppercase letters

```
[a-zA-Z]
```

Both

Negation

```
[^abc]
```

Matches everything except:

a
b
c

5. Common Regex Shortcuts

Pattern Meaning

\d	Digit
\D	Non-digit
\s	Whitespace
\S	Non-whitespace
\w	Word character
\W	Non-word character

Example

```
regex = r"[aA]\d"
```

Matches:

A4
a7

6. Groups

Groups allow multiple characters to act as one unit.

Example

(abc)

Alternative Matching

(abc|def)

Matches:

abc

def

7. Repetition

+

One or more

.\.txt

Matches: data.txt, notes.txt

Zero or more

.*\.txt

Matches: .txt , data.txt

?

Optional

colou?r

Matches: color , colour

Exact Count

(ha){2}

Matches:

haha

Range Count

(ha){2,5}

Matches:

haha

hahaha

hahahaha

8. Hex Color Validation

Six Character Hex

```
^#(\d|[A-Fa-f]){6}$
```

Matches:

#FF1234

#12ABEF

Three or Six Character Hex

```
^#((\d|[A-Fa-f]){3}){1,2}$
```

Matches:

#ABC

#AABBCC

9. Python re Module Function

```
import re
```

search()

Find first occurrence.

```
re.search(pattern, text)
```

Example:

```
import re
```

```
text = "Python Regex"
```

```
result = re.search("Regex", text)
```

```
print(result)
```

match()

Checks only beginning.

```
re.match(pattern, text)
```

Example:

```
re.match("Python", "Python Regex")
```

✓ Match

fullmatch()

Entire string must match.

```
re.fullmatch(r"\d+", "12345")
```

✓ Match

```
re.fullmatch(r"\d+", "123abc")
```

✗ No match

findall()

Find all matches.

```
re.findall(r"\d", "A1B2C3")
```

Output:

```
['1', '2', '3']
```

sub()

Replace text.

```
re.sub(r"\d", "*", "A1B2C3")
```

Output:

```
A*B*C*
```

subn()

Returns replacement count.

```
re.subn(r"\d", "*", "A1B2C3")
```

Output:

```
('A*B*C*', 3)
```

split()

Split string using regex.

```
re.split(r"\s+", "Python Regex Playbook")
```

Output:

```
['Python', 'Regex', 'Playbook']
```

escape()

Escapes special characters.

```
re.escape("file.txt")
```

Output:

```
file\.txt
```

compile()

Reuse regex efficiently.

```
pattern = re.compile(r"\d+")
```

10. Match Object

Returned by:

```
re.search()
```

```
re.match()
```

```
re.fullmatch()
```

Useful Methods

group()

`match.group()`

Matched text

start()

`match.start()`

Start position

end()

`match.end()`

End position

span()

`match.span()`

Tuple of start and end

Example

```
import re
```

```
match = re.search(r"\d+", "Age 25")
```

```
print(match.group())
```

```
print(match.start())
```

```
print(match.end())
```

```
print(match.span())
```

Output:

25

4

6

(4, 6)

11. Regex Flags

Case Insensitive

re.I

or

re.IGNORECASE

Example:

```
re.search("python", "Python", re.I)
```

✓ Match

Multiline

re.M

or

re.MULTILINE

Makes:

^

\$

apply to every line.

Dotall

re.S

or

re.DOTALL

Makes:

.

match newlines too.

12. Searching Patterns in Files

Extract Product Codes

Example from course:

```

import re

with open("sample.txt", "r") as f:
    text = f.read()

pattern = r"\b[A-Z]{2}\d{2}-[A-Z]{2}\d{2}\b|\b\d{3}-\d{3}\b"

matches = re.findall(pattern, text)

print("All product codes:", matches)

```

13. User Input Validation

Username Validation

```

import re

def valid_username(username):
    pattern = r"^[A-Za-z0-9_]{4,15}$"
    return re.fullmatch(pattern, username)

```

Numeric Validation

```

import re

def is_numeric(value):
    return re.search(r"^\d+$", value)

```

14. Email Validation

Course pattern:

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

Example:

```

import re

email = "test@gmail.com"

if re.fullmatch(
    r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",

```

```
email
):
    print("Valid")
```

15. Lookahead and Lookbehind

Sample Text

```
text = "aB3cD4eF5gH6"
```

Lookbehind

Uppercase letter preceded by lowercase.

```
pattern = r"(?<=[a-z])[A-Z]"
```

Lookahead

Uppercase preceded by lowercase and followed by digit.

```
pattern = r"(?<=[a-z])[A-Z](?=\d)"
```

Example

```
import re
```

```
text = "aB3cD4eF5gH6"
```

```
matches = re.findall(
    r"(?<=[a-z])[A-Z](?=\d)",
    text
)
```

```
print(matches)
```

Output:

```
['B', 'D', 'F', 'H']
```

16. Data Cleaning with Regex

Standardize Phone Numbers

Example:

```
import re

phone = "(123) 456-7890"

clean = re.sub(r"\D", "", phone)

print(clean)
```

Output:

1234567890

Remove Punctuation

```
import re

text = "Hello, World!"

clean = re.sub(r"[^\w\s]", "", text)

print(clean.lower())
```

Output:

hello world

17. Redacting Sensitive Information

Example:

```
import re

text = "My credit card is 1234-5678-9012-3456"

redacted = re.sub(
    r"\d{4}-\d{4}-\d{4}-\d{4}",
    "****_****_****_****",
    text
```

)

```
print(redacted)
```

Output:

My credit card is ****_****_****_****